Universidade Estadual da Paraíba

Centro de Ciências e Tecnologia - CCT

Departamento de Computação

Curso de Ciência da Computação

Disciplina: Compiladores

Análise Léxica

Professora: Kézia de Vasconcelos Oliveira Dantas
kezia.vasconcelos@servidor.uepb.edu.br

Analisador Léxico

- É a primeira fase de um processo de compilação.

- Sua função é fazer a leitura do programa fonte, caractere a caractere, agrupar os caracteres em lexemas e produzir uma sequência de símbolos léxicos conhecidos como tokens.

- Deve interagir com a tabela de símbolos inserindo informações de alguns tokens, como por exemplo os identificadores.

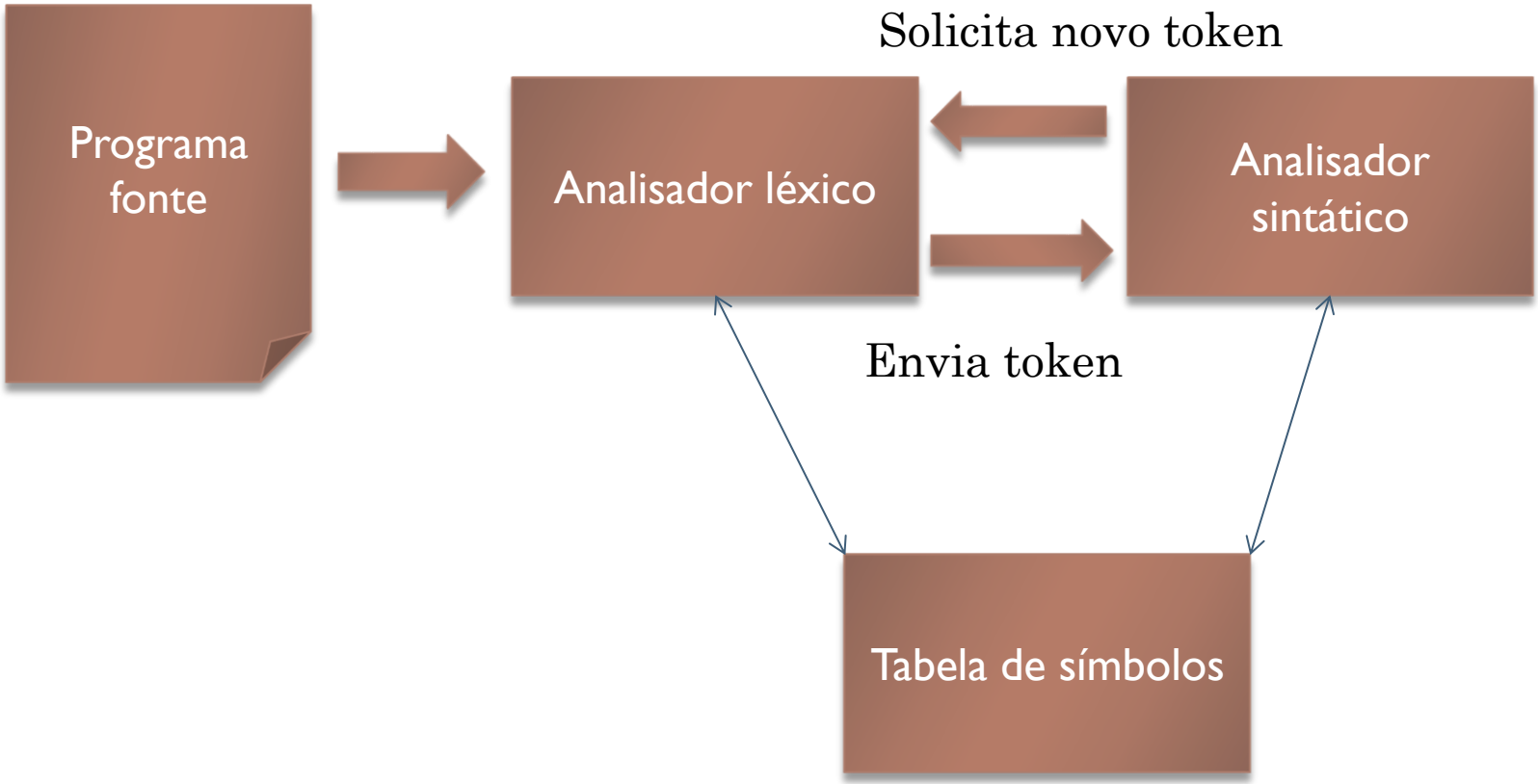
Papel do Analisador Léxico

- O analisador léxico interage diretamente com o analisador sintático.

- A nível de implementação a analise léxica normalmente é uma sub-rotina da análise sintática formando um único passo.

- A separação da análise léxica da análise sintática facilita o projeto e torna o compilador mais eficiente e portável.

Papel do Analisador Léxico



Papel do Analisador Léxico

- Um analisador léxico executa tarefas como:
  - Contar as linhas de um programa
  - Eliminar comentários
  - Contar a quantidade de caracteres de um arquivo
  - Tratar espaços

- A análise léxica pode ser dividida em duas etapas:
  - Escandimento: remoção de comentários e espaços em branco.
  - Análise léxica propriamente dita: geração dos tokens.

Analisador Léxico - Terminologia

- Termos essenciais ao analisador léxico:
  - Padrão: é a forma que os lexemas de uma cadeia de caracteres pode assumir.
    - Ex: no caso de palavras reservadas é a sequência de caracteres que formam a palavra reservada
  - Lexema: é uma sequência de caracteres reconhecidos por um padrão.
  - Token: é um par constituído de um nome e um valor de atributo opcional. O nome de um token é um símbolo que representa a unidade léxica. Por exemplo: palavras reservadas; identificadores; números, etc.

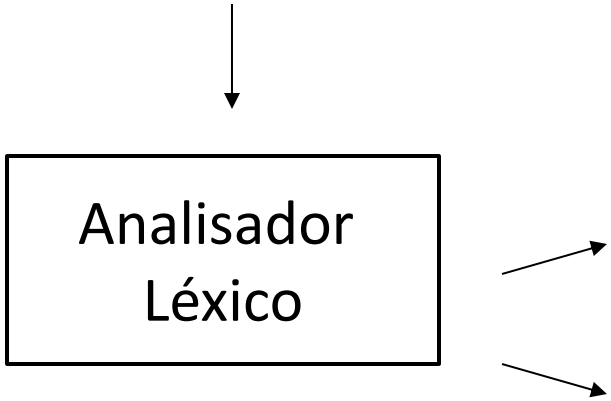
Analisador Léxico - Terminologia

- Termos essenciais ao analisador léxico:

Token	Lexemas Exemplo	Descrição informal do padrão
if	if	if
relação	<, <=, =, >, >=	< ou <= ou = ou > ou >=
id	pi, contador, varSoma	Letra seguida por letras ou dígitos
num	3.1416, 0, 6.02E23	Qualquer constante numérica
literal	"string qualquer"	Quaisquer caracteres entre aspas, exceto aspas

Analisador Léxico – Atributos de tokens

```
position = initial + rate * 60
```



<**identificador**, 1>, <=>,
<**identificador**, 2>, <+>,
<**identificador**, 3>, <*>,
<**number**, 60>

Tabela de Símbolos

	nome	**tipo**	
1	`position`	-	...
2	`initial`	-	
3	`rate`	-	
...			

Analisador Léxico - Atributos de tokens

- Quais as classes de tokens que podem ser reconhecidos em uma linguagem de programação como C ?
  - Palavras reservadas: if else while do
  - Identificadores
  - Operadores relacionais: < > <= >= == !=
  - Operadores aritméticos: + * / -
  - Operadores lógicos: && || & | !
  - Operador de atribuição: =
  - Delimitadores: ; ,
  - Caracteres especiais: ( ) [ ] { }
  - Numeros

Analisador Léxico - Atributos de tokens

- Exemplo 1: E = M * C ** 2

  - Sequência de tokens:

    <id, 1>, <=> ,<id, 2>,<*>, <id,3>, <**>, <numero, 2>

  - Temos os seguintes tipos de tokens:
    - <id, 1> : apontador 1 da tabela de símbolos e classe do token id.
    - <=> operador de atribuição, sem necessidade de um valor para o atributo;
    - <numero, 2> : token número, com valor para o atributo 2 indicado o valor do número (constante numérica).

Analisador Léxico - Atributos de tokens

- Exemplo 2: total = entrada * saida() + 2

  - Sequência de tokens:

    <id, 1> <relop,= > <id, 2> <ariop, *> <id,3>, <delim,(>, <delim,)>, < ariop, +>, <numero, 2>

  - Temos os seguintes tipos de tokens :
    - <id, 1> : apontador 1 da tabela de símbolos e classe do token id;
    - <relop,=> operador de atribuição, sem necessidade de um valor para o atributo;
    - <delim, (>: Delimitador ;
    - <numero, 2> : token número, com valor para o atributo 2 indicado o valor do número (constante numérica).

Analisador Léxico - Atributos de tokens

- Exemplo 3:

```
while indice < 10 do
        indice:= total + indice;
```

  - Sequência de tokens gerados:
    - <while,>
    - <id,1>
    - <relop,<>
    - <numero,10>
    - <do,>
    - <id,1>
    - <relop,:=>
    - <id,2>
    - <ariop,+>
    - <id, 1>
    - <delim,; >

Erros Léxicos

- A análise léxica é muito prematura para identificar alguns erros de compilação, veja o exemplo abaixo:

fi (a == “123”) ...

- A única possibilidade de ocorrer erro é aparecer um caractere que não pertence ao alfabeto da linguagem.
  - O projetista esqueceu ou o usuário cometeu erro.

Recuperação de Erros Léxicos

- Na ocorrência de erros o analisador léxico pode parar ou entrar em loop infinito. A modalidade de pânico pode ser usada para recuperar erros léxicos
  - Estratégia mais simples;
  - Remoção de sucessivos caracteres até o reconhecimento de um token válido.

- Outras formas de recuperar erros:
  1. Remover um caractere da entrada restante
  2. Inserir um caractere que falta na entrada restante
  3. Substituir um caractere incorreto por outro correto
  4. Trocar dois caracteres adjacentes

Recuperação de Erros Léxicos

- Estas transformações seriam computadas na tentativa de reparar a entrada.
  - Estratégia 1: verificar se o prefixo da entrada restante pode ser transformado em um lexema válido por uma única transformação.
    - Na prática a maioria dos erros léxicos envolvem um caractere
  - Estratégia 2: encontrar o menor número de transformações necessárias para que o programa fonte seja formado apenas por lexemas válidos.
    - Na prática esta estratégia demanda muito esforço.

Especificação dos Tokens

- Cadeias e Linguagens
- Operações em Linguagens
- Expressões Regulares

Cadeias e Linguagens

- **Alfabeto ou classe de caracteres**: qualquer conjunto finito de símbolos.
  - Alfabeto binário {0,1}
  - ASCII
- **Cadeia, sentença ou palavra:** nome dada a uma sequência finita de símbolos retiradas de um alfabeto
  - Ex: banana, 010101000001
  - O comprimento de um palavra, denotado por |s|, corresponde ao número de símbolos requeridos para sua construção
- **Linguagem:** denota qualquer conjunto de cadeias sobre algum alfabeto fixo
  - Ǿ, conjunto de todos os programas Java sintaticamente bem formados, etc

Operações em Linguagens

- **Prefixo:** cadeia obtida pela remoção de zero ou mais símbolos no fim da cadeia. Ex: *ban* é um prefixo de *banana*.

- **Sufixo:** cadeia obtida pela remoção de zero ou mais símbolos no inicio da cadeia. Ex: n*ana* é um sufixo de *banana*.

- **Subcadeia:** cadeia obtida pela remoção de um prefixo e de um sufixo. Ex: *nan*.

Operações em Linguagens

- **União:** qualquer cadeia pertencente a um dos dois conjuntos. L *U* M = { *s*|*s está em L ou s está em M*} sendo L e M linguagens quaisquer.

- **Concatenação:** LM = {*st*|*s está em L e t está em M*}.

- **Fechamento Kleene (L*):** zero ou mais concatenações de L.

- **Fechamento positivo (L^+):** uma ou mais concatenações de L.

Operações em Linguagens - exemplos

- L U D

  Conjunto de letras e dígitos.
- LD

  Conjunto de cadeias formadas por uma letra seguida por um dígito. Ex: a1
- L^4

  Conjunto de cadeias formadas por 4 letras. Ex: abcd
- L*

  Conjunto de cadeias formadas por zero ou mais letras. Ex: a, ab, bb, bbc, ...
- L (L U D)*

  Conjunto de todas as cadeias de letras e dígitos que comecem com uma letra
- D^+

  Conjunto de todas as cadeias de um ou mais dígitos

Expressões Regulares

- Notação especial para definição de cadeias de uma linguagem
- Identificador Pascal
  - letra (letra|dígito)*
  - Caractere | é igual a *ou*
  - * significa zero ou mais instâncias
  - A justaposição de letras significa concatenação destas
  - Ex:
    - a|b {a,b}
    - (a|b)(a|b) {aa, ab, ba, bb}
    - a* {ε, a, aa, aaa, ...}
    - (a|b)*
- Se duas expressões regulares r e s denotam a mesma linguagem, dizemos que são equivalentes e representamos r = s. Ex: (a|b) = (b|a)

Expressões Regulares

- Definições regulares
  - Expressões regulares podem ser nomeadas e estes nomes podem ser utilizados para definição de novas expressões
  - Ex:

letra : A|B|...|Z|a|b|...|z

digito : 0|1|...|9

id : **letra(letra|digito)***

Expressões Regulares

- $[a\text{-}z]^+$

  denota o conjunto { a, aa, aaa, ..., b, bb, ..., aba, ... }, ou seja, cadeias formadas de caracteres entre a e z

- $[a\text{-}z][0\text{-}9]^+$

  denota o conjunto { a0, a123, d25, ... }, ou seja, cadeias formadas por um caractere de a a z seguidas por um ou mais dígitos

- $[w|W][h|H][i|I][l|L][e|E]$

  denota o conjunto { while, While, wHile, WHile, ... }

- $[0\text{-}9]^+([.][0\text{-}9]^+)^?$

  denota o conjunto de todos as sequências de dígitos, mais o conjunto das sequências de dígitos seguidas por um ponto e outra sequência de dígitos

Expressões Regulares

- [^a]
  denota todos os caracteres exceto a
- [^x-y]
  denota todos os caracteres exceto os que estão entre x e y
- Validação de CEP ou CPF?
- Comentários?

Especificação Formal de Linguagens – Linguagens Regulares (LR)

- LR é a classe das linguagens mais simples

- O estudo de LR pode ser abordado através de três diferentes formalismos:
  - Axiomático ou gerador: Gramática Regular
  - Denotacional: Expressão Regular
  - Operacional ou reconhecedor: Autômato Finito
    - Tipos: NFAs e DFAs
    - Implementam qualquer expressão regular

Autômato Finito

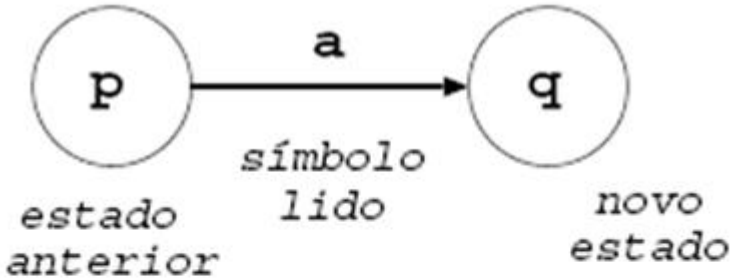
- Definição: Um autômato finito M é uma 5-upla:

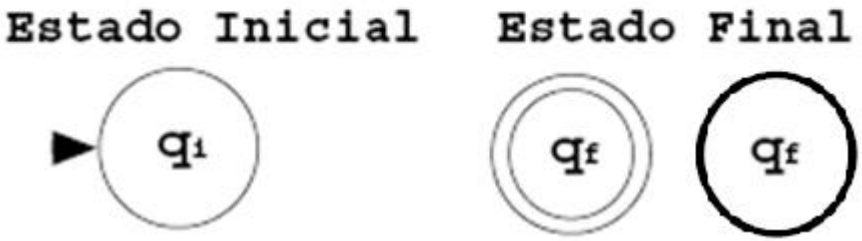
$$M = (Q, \Sigma, \delta, q_0, F), \text{ onde:}$$

  - Q - conjunto finito de estados
  - Σ - conjunto finito de símbolos chamado de alfabeto
  - δ - função de transição do autômato
  - $q_0 \in Q$ - estado inicial
  - $F \subseteq Q$ - conjunto de estados finais

Autômatos Finitos

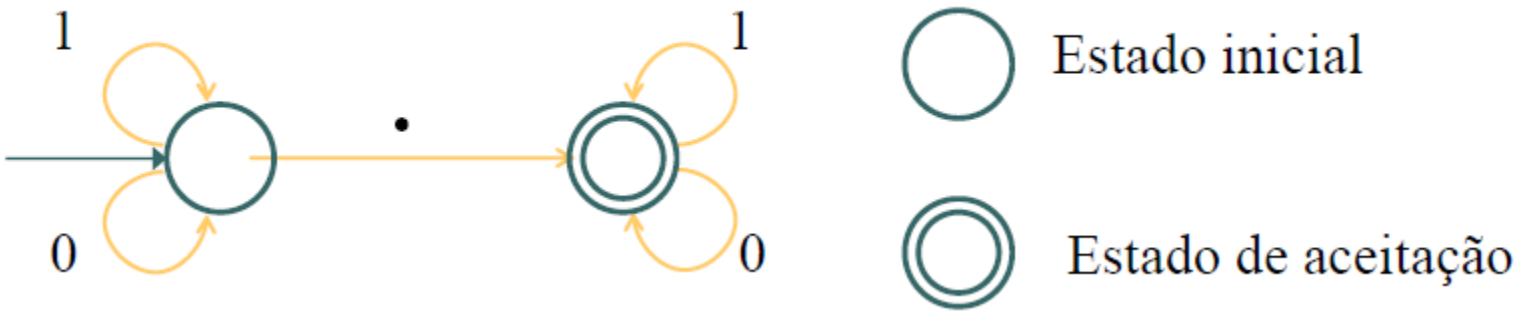
- Um AF pode ser representado na forma de um diagrama no qual:





Autômatos Finitos

- Exemplo: (0 | 1)* . (0 | 1)*

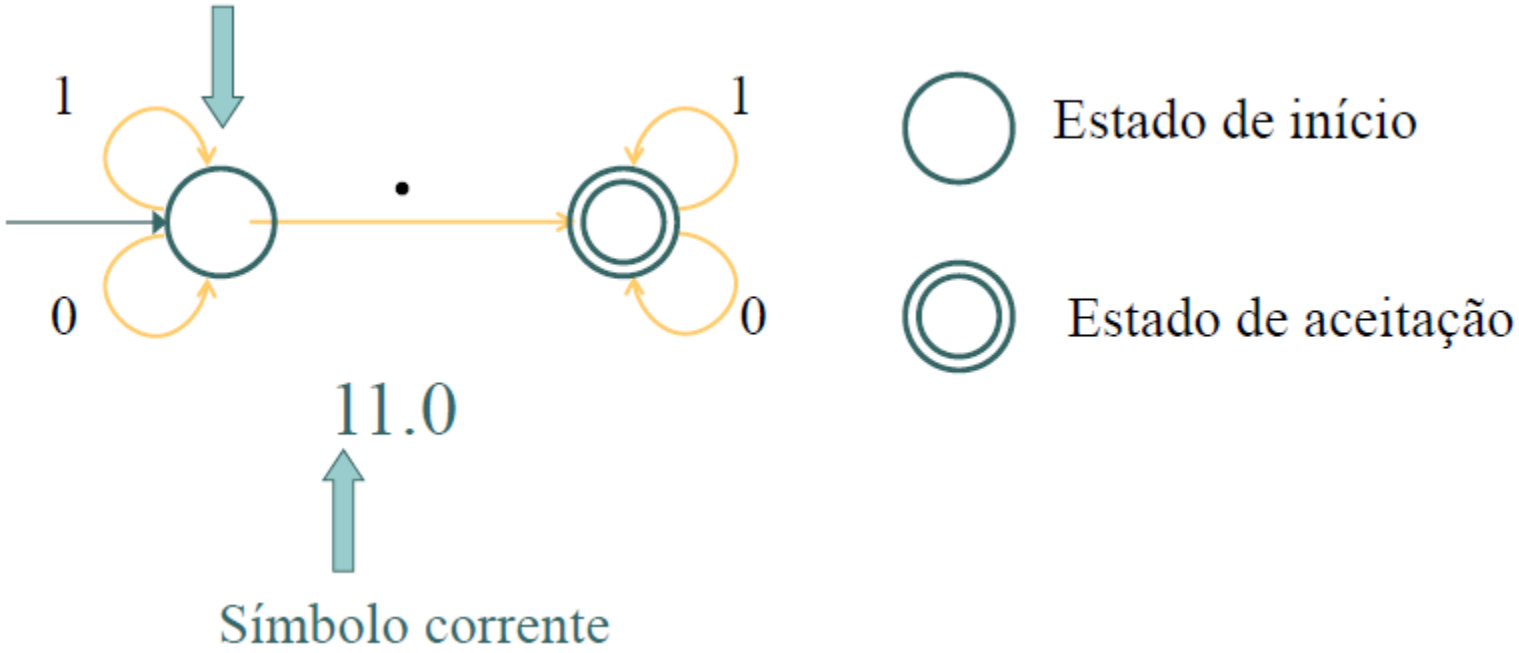


Autômatos Finitos

- Uma palavra $w \in \Sigma*$ é dita ser aceita por um AF quando,partindo do estado inicial, forem lidos todos os símbolos de w e efetuadas as correspondentes transições de modo que, ao ler o último símbolo, o AF para em um estado final.

Autômatos Finitos

- Exemplo



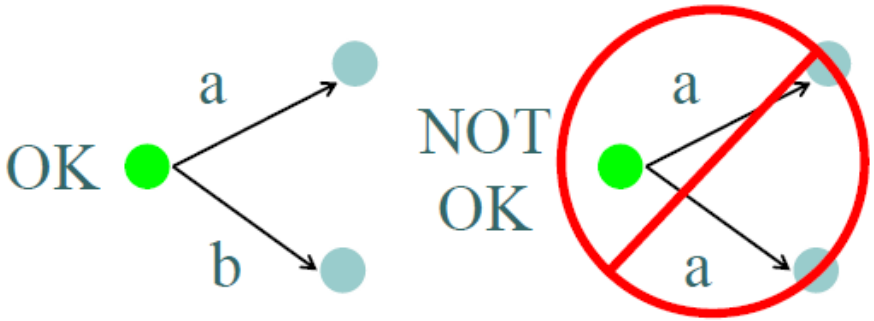
Autômatos Finitos

- DFA: Autômato Finito Determinístico
  - De um determinado estado, a ocorrência de um símbolo não pode ter mais do que um arco, e por isso não pode levar a estados diferentes
  - $\delta : Q \times \Sigma \rightarrow Q$

- NFA: Autômato Finito Não-Determinístico
  - De um determinado estado, a mesma ocorrência pode conduzir a estados distintos
  - $\delta : Q \times \Sigma \rightarrow 2^Q$

Autômatos Finitos

- DFA
  - Sem transições ε
  - No máximo uma transição de cada estado para cada símbolo



- NFA – nenhuma destas restrições

Observações

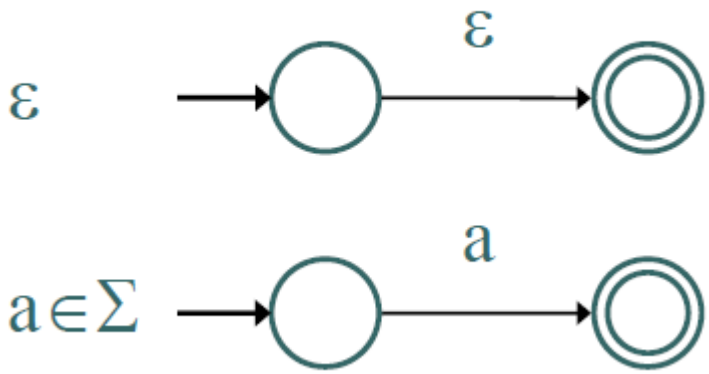
- Expressões regulares: mecanismo para gerar as palavras da linguagem
- Autômatos: mecanismo para reconhecer se uma palavra específica pertence à linguagem
- Abordagem standard
  - Usar expressões regulares para definição da linguagem
  - Tradução automática para autômatos para a implementação do analisador léxico

Da Expressão Regular ao Autômato

- Construção por indução estrutural
- Dada uma expressão regular r arbitrária
- Assumir que podemos convertê-la para um autômato com
  - Um estado inicial
  - Um estado de aceitação

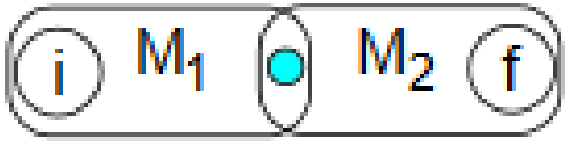
Da Expressão Regular ao Autômato

- Construtores Básicos
  - Um símbolo a do alfabeto da linguagem Σ



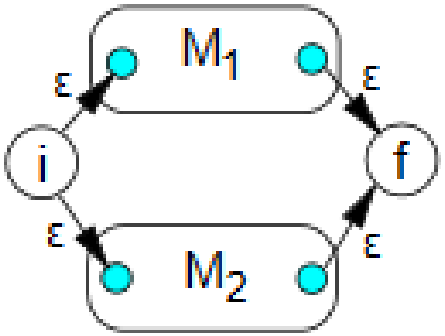
Da Expressão Regular ao Autômato

- r1r2
  - Concatenação RS, considere que M1 reconhece R e M2 reconhece S
  - o estado final de M1 é combinado com o estado inicial de M2



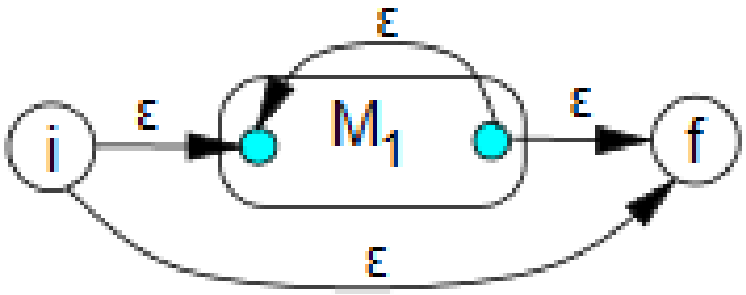
Da Expressão Regular ao Autômato

- r1|r2
  - União, com M1 e M2 como no caso anterior
  - Um novo estado inicial é introduzido, com transições pela string vazia para os estados iniciais de cada uma das máquinas M1 e M2. Similarmente, um novo estado final é atingido com transições pela string vazia a partir dos estados finais dessas máquinas



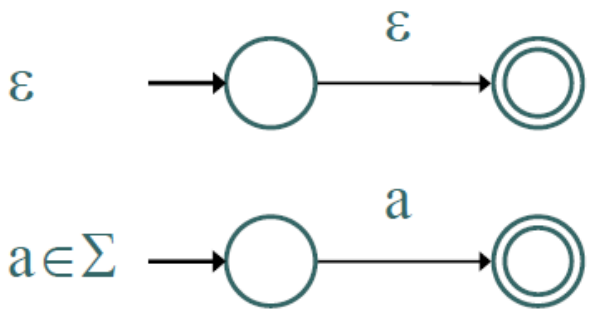
Da Expressão Regular ao Autômato

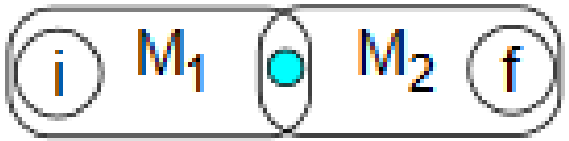
- r*
  - Considere que M1 reconhece R
  - Novos estados inicial e final são introduzidos, com transições pela string vazia para reconhecer zero ocorrências (direto do novo estado inicial para o novo estado final) e várias ocorrências (volta do estado final de M1 para o estado inicial de M1)



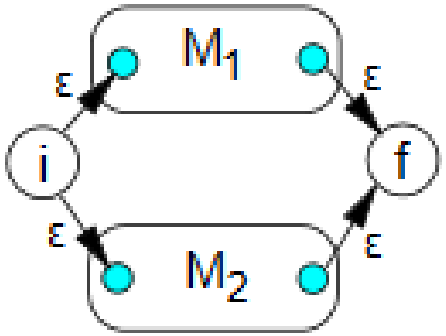
Da Expressão Regular ao Autômato

- Regras de conversão

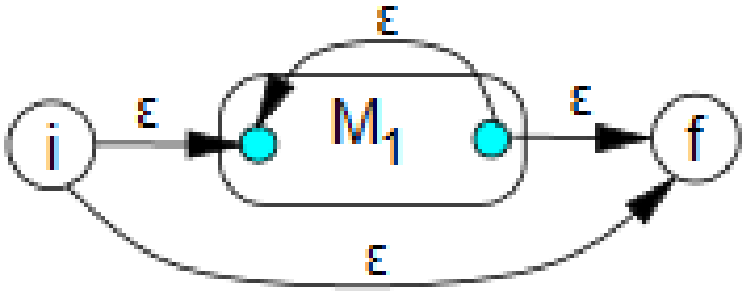




r1r2



r1|r2



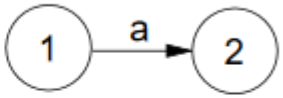
r*

Da Expressão Regular ao Autômato

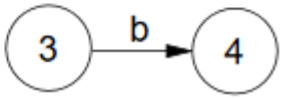
- Exemplo: (a|b)*abb

$$R_1 = a \qquad R_2 = b$$
$$R_3 = R_1|R_2 \qquad R_4 = R_3{}^*$$
$$R_5 = R_4R_1 \qquad R_6 = R_5R_2$$
$$R = R_6R_2$$

Para reconhecer R_1,



Similarmente, para reconhecer R_2,

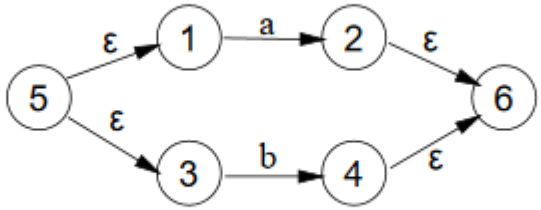


Da Expressão Regular ao Autômato

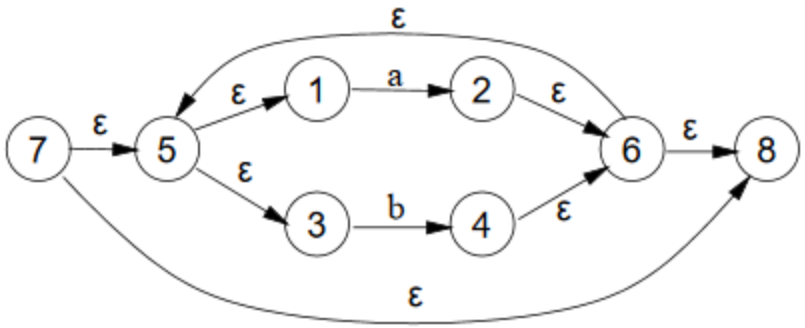
- Exemplo: (a|b)*abb

$$R_1 = a \qquad R_2 = b$$
$$R_3 = R_1|R_2 \qquad R_4 = R_3{}^*$$
$$R_5 = R_4R_1 \qquad R_6 = R_5R_2$$
$$R = R_6R_2$$

O autômato para reconhecer R_3 é construído pela combinação-alternativa de R_1 e R_2,



Para reconhecer R_4, constrói-se o autômato que reconhece 0 ou mais ocorrências de R_3:

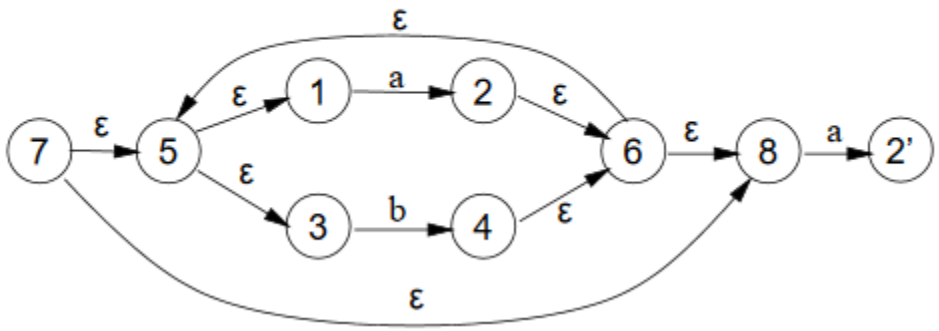


Da Expressão Regular ao Autômato

- Exemplo: (a|b)*abb

$$R_1 = a \qquad R_2 = b$$
$$R_3 = R_1|R_2 \qquad R_4 = R_3{}^*$$
$$R_5 = R_4R_1 \qquad R_6 = R_5R_2$$
$$R = R_6R_2$$

O autômato para reconhecer R_5 é formado pela combinação de concatenação de R_4 com um autômato equivalente àquele que reconhece R_1,

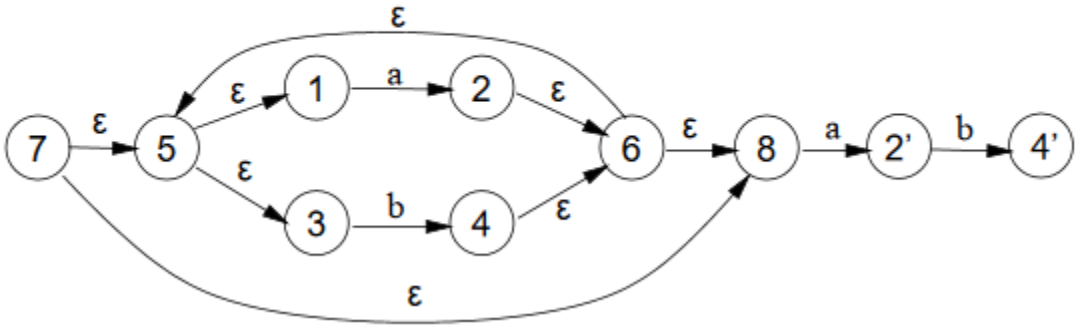


Da Expressão Regular ao Autômato

- Exemplo: (a|b)*abb

$$R_1 = a \qquad R_2 = b$$
$$R_3 = R_1|R_2 \qquad R_4 = R_3{}^*$$
$$R_5 = R_4R_1 \qquad R_6 = R_5R_2$$
$$R = R_6R_2$$

Similarmente, para reconhecer R_6 é preciso combinar o autômato que reconhece R_5 com um equivalente àquele que reconhece R_2,

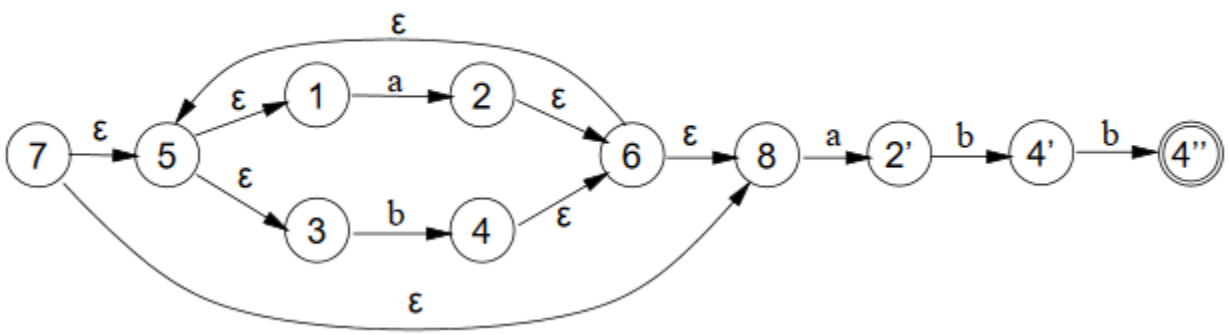


Da Expressão Regular ao Autômato

- Exemplo: (a|b)*abb

$$R_1 = a \qquad R_2 = b$$
$$R_3 = R_1|R_2 \qquad R_4 = R_3{}^*$$
$$R_5 = R_4R_1 \qquad R_6 = R_5R_2$$
$$R = R_6R_2$$

A expressão completa é uma concatenação de R_6 com um autômato que reconhece b, equivalente à R_2:



Da Expressão Regular ao Autômato

- Exercícios: Converta as seguintes expressões regulares para autômatos finitos

  a. (0 | 1)* . (0 | 1)*
  b. (a*|b*)*
  c. (a|b)*abb(a|b)*

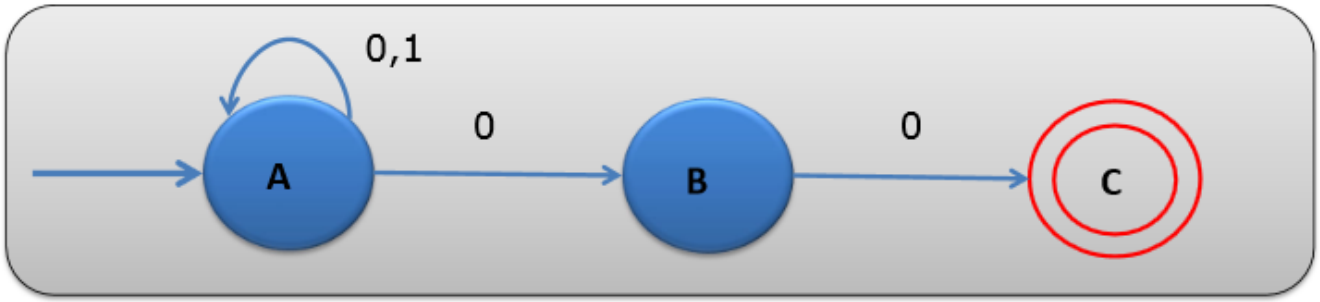
Da Expressão Regular ao Autômato

- A conversão de uma expressão regular para um autômato baseada nas regras apresentadas produz um NFA

- Há interesse em ter um DFA para facilitar o algoritmo de reconhecimento

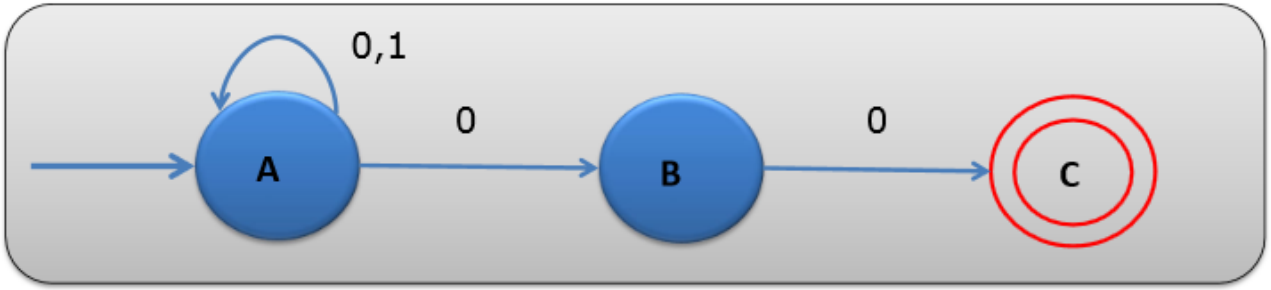
- Pode-se converter do NFA para o DFA
  - DFA pode ser exponencialmente maior do que o NFA
  - A simplificação do DFA resultante envolve a eliminação de estados equivalentes

Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}		

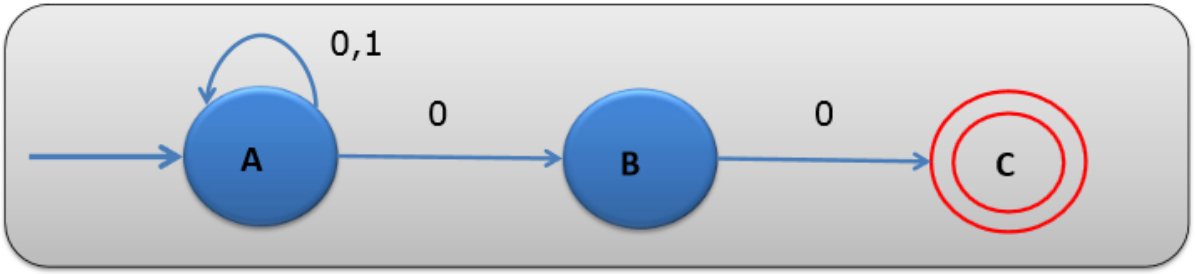
Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}	{A , B}	

A entrada do símbolo 0 pode ocasionar o deslocamento para os estados A ou B: {A , B}

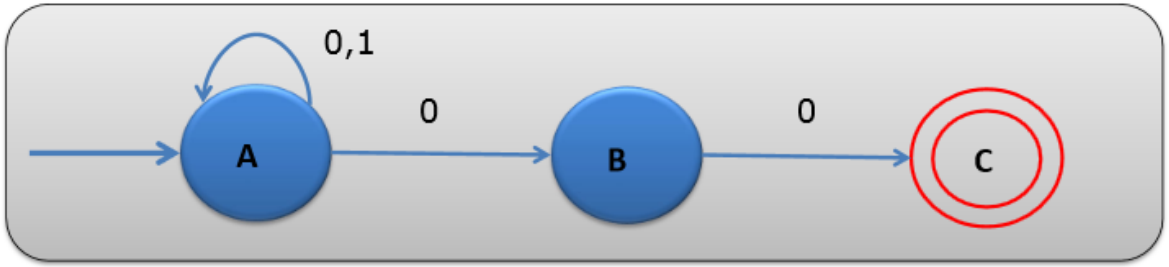
Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}	{A , B}	{A}

Dos conjuntos gerados após a entradas dos símbolos 0 e 1, deve se analisar aqueles que não foram tratados ainda, neste caso o conjunto {A , B}

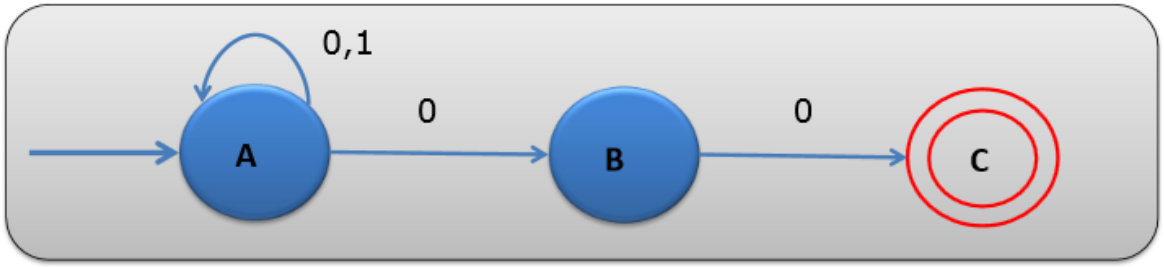
Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}	{A , B}	{A}
	{A , B}		

Deve-se analisar a transição da união {A , B}, para cada símbolo de entrada

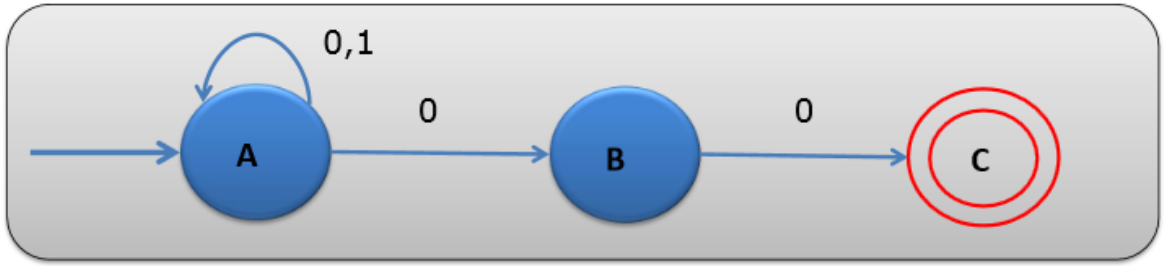
Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}	{A , B}	{A}
	{A , B}	{A,B,C}	

Para o estado A, com o símbolo de entrada 0, é deslocado para os estados A ou B.
Para o estado B, com o símbolo de entrada 0, é deslocado para o estado C.

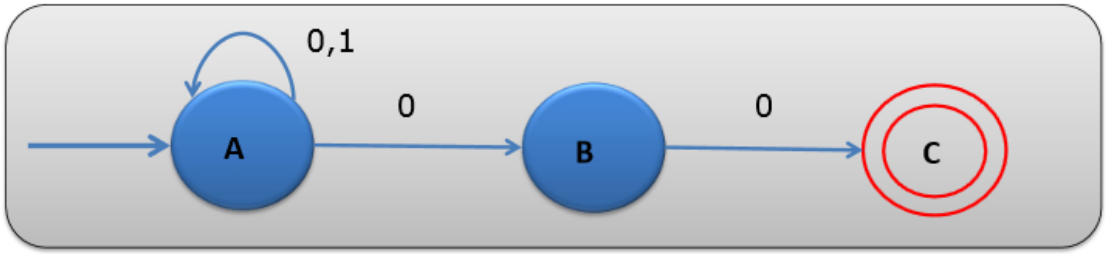
Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}	{A , B}	{A}
	{A , B}	{A,B,C}	{A}

Para o estado A, com o símbolo de entrada 1, é deslocado para os estados A.
Para o estado B, com o símbolo de entrada 1, não existe deslocamento de estado.

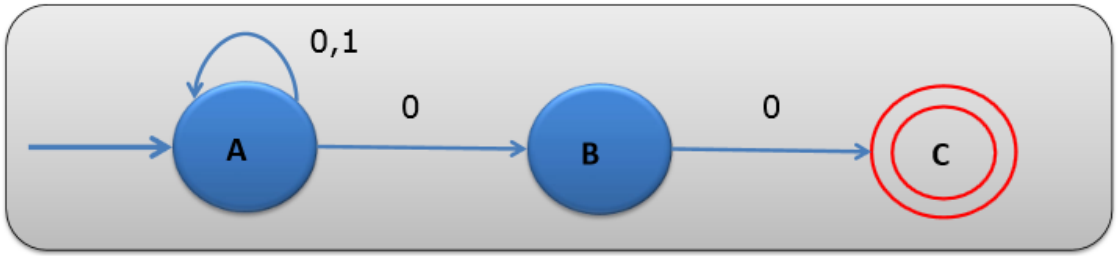
Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}	{A , B}	{A}
	{A , B}	{A , B , C}	{A}

Dos conjuntos formados deve se analisar aquele que não foi tratado ainda. neste caso {A . B . C}

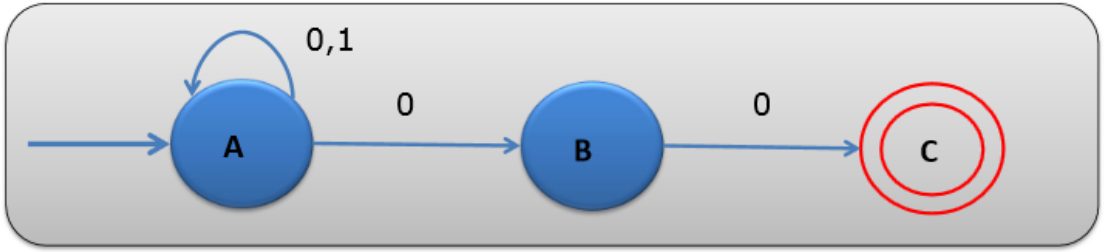
Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}	{A , B}	{A}
	{A , B}	{A , B , C}	{A}
	{A , B , C}		

Realizando a análise para os símbolos de entrada 0 e 1

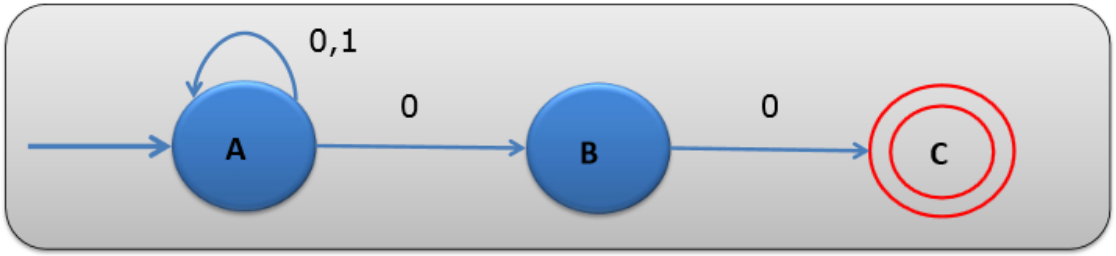
Conversão de NFA para AFD



	Estado	Símbolo 0	Símbolo 1
Estado Inicial	{A}	{A , B}	{A}
	{A , B}	{A , B , C}	{A}
	{A , B , C}	{A , B , C}	{A}

Não tem novos conjuntos gerados se finaliza o procedimento de transição.

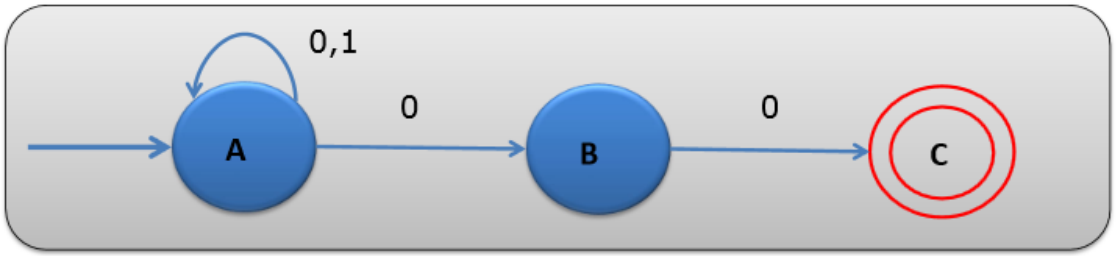
Conversão de NFA para AFD



	Estado	Novos Estados	Símbolo 0	Símbolo 1
Estado Inicial	{A}	**e0 (inicial)**	{A , B}	{A}
	{A , B}	**e1**	{A , B , C}	{A}
	{A , B , **C**}	**e2 (final)**	{A , B , C}	{A}

O estado final é aquele que contem o conjunto com o estado final do AFND original, ou seja "**C**"

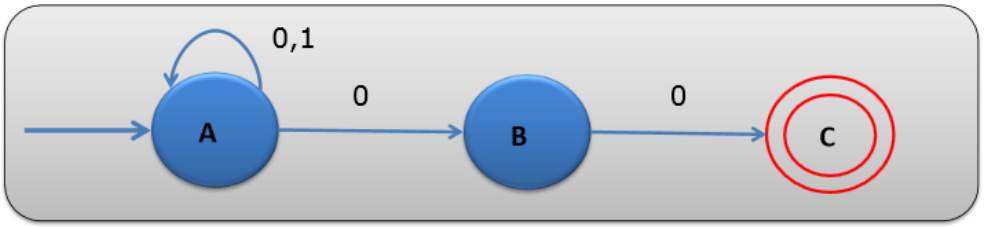
Conversão de NFA para AFD



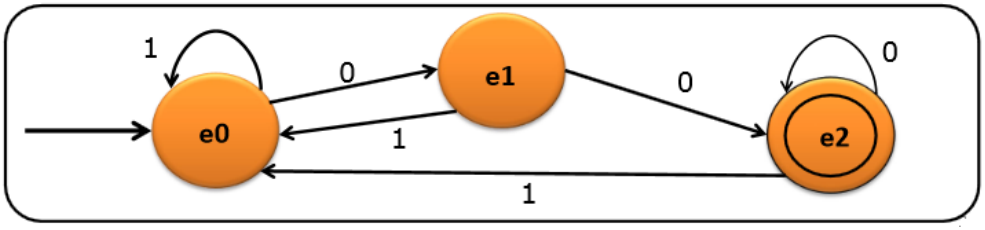
	Estado	Novos Estados	Símbolo 0	Símbolo 1
Estado Inicial	{A}	**e0 (inicial)**	{A , B} **(e1)**	{A} **(e0)**
	{A , B}	**e1**	{A , B , C} **(e2)**	{A} **(e0)**
	{A , B , **C**}	**e2 (final)**	{A , B , C} **(e2)**	{A} **(e0)**

Renomeando os estados

Conversão de NFA para AFD



	Estado	Novos Estados	Símbolo 0	Símbolo 1
Estado Inicial	{A}	**e0 (inicial)**	{A , B} **(e1)**	{A} **(e0)**
	{A , B}	**e1**	{A , B , C} **(e2)**	{A} **(e0)**
	{A , B , **C**}	**e2 (final)**	{A , B , C} **(e2)**	{A} **(e0)**



Implementação do Analisador Léxico

- Definição das expressões regulares
- Tradução das expressões regulares para um NFA
  - Do NFA para o DFA
- Implementação em software do DFA

Implementação do Analisador Léxico

- Três formas de implementação manual do código:
  - Ad hoc – tem sido muito usada
  - Código que reflete diretamente um AF
  - Uso de Tabela de Transição e código genérico
- Uso do Lex (gerador de AL) ou outro compiler como o JavaCC

Implementação do Analisador Léxico

- Em implementações manuais do AL, é comum reconhecer uma palavra reservada como identificador
  - Depois fazer a checagem numa tabela de palavras reservadas
    - Solução simples e elegante

- A eficiência de um AL depende da eficiência da checagem na tabela
  - Em compiladores reais não são implementadas com busca linear
    - Usa-se busca binária ou hashing, pois temos todas as palavras de antemão

Implementação do Analisador Léxico

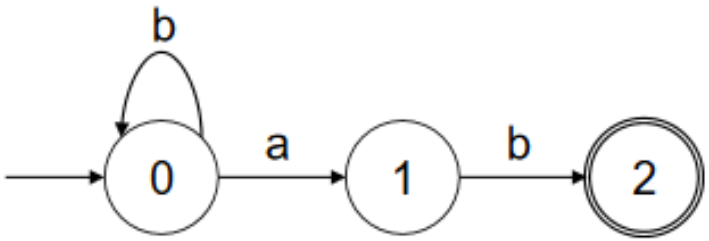
- Há um grande cuidado na implementação da variável token, que recebe os tokens do programa
  - Para certos casos como símbolos especiais basta definir como string de tamanho 2; para palavras reservadas não deve passar de 10
  - Mas como fazer para identificadores, strings, números???
  - Identificadores preocupam, pois eles ficam guardados na Tabela de Símbolos e reservar 256 caracteres para cada um pode ser abusivo em termos de espaço
    - Uma saída é usar alocação dinâmica para alocar o tamanho exato de cada token.

Implementação do Analisador Léxico – Solução Ad hoc

Mantém o estado implicitamente, indicado nos comentários

```
{início – estado 0}
c:=próximo_caractere()
se (c='b') então
        c:=próximo_caractere()
        enquanto (c=b) faça
                c:=próximo_caractere()
{ estado 1}
        se (c='a') então
                c:=próximo_caractere()
{estado 2}
        se (c='b') e (acabou cadeia de entrada) então retornar "cadeia aceita"
                senão retornar "falhou"
        senão retornar "falhou"
{ estado 1}

senão se (c='a') então
        c:=próximo_caractere()
 {estado 2}
        se (c='b') e (acabou cadeia de entrada) então retornar "cadeia aceita"
        senão retornar "falhou"
senão retornar "falhou"
```



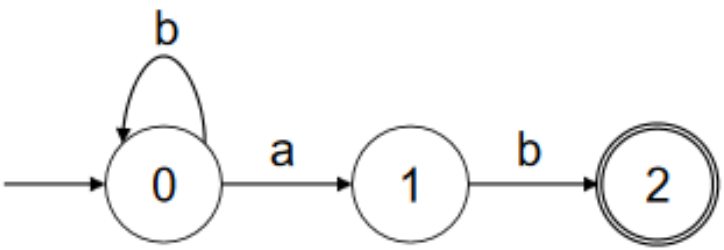
Implementação do Analisador Léxico – Solução Ad hoc

- Simples e fácil
- Mantém o estado implicitamente, indicado nos comentários
- Razoável se não houver muitos estados, pois a complexidade do código cresce com o aumento do número de estados
- Problema: por ser ad hoc, se mudar o AF temos que mudar o código

Implementação do Analisador Léxico – Solução: Incorporação das transições no código do programa

- Uso de uma variável para manter o estado corrente

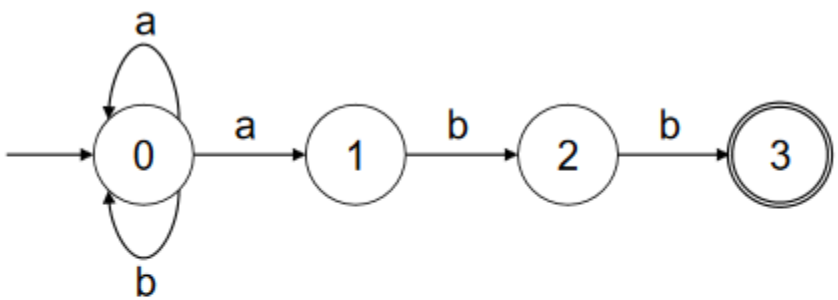
```
s:=0 {uso de uma variável para manter o estado corrente}
enquanto s = 0 ou 1 faça
    c:=próximo_caractere()
    caso (s) seja
        0: se (c=a) então s:=1
           senão se (c=b) então s:=0
           senão retornar "falhou"; s:= outro
        1: se (c=b) então s:=2
           senão retornar "falhou"; s:= outro
    fim caso
fim enquanto
Se s = 2 então "aceitar" senão "falhou";
```



Implementação do Analisador Léxico – Solução: Incorporação das transições no código do programa

- Reflete diretamente o AF
- Mas cada código é ainda diferente, caso mude o AF
- Uma outra opção seria usar um código genérico e expressar o AF como estrutura de dados

Implementação do Analisador Léxico – Solução: Métodos Dirigidos por Tabela



Estado	Símbolo de entrada	
	a	b
0	{0,1}	{0}
1	---	{2}
2	---	{3}

Problema: tabela não indica estados de aceitação nem quando não se consome entrada. Temos que estende-la

Implementação do Analisador Léxico – Solução: Métodos Dirigidos por Tabela

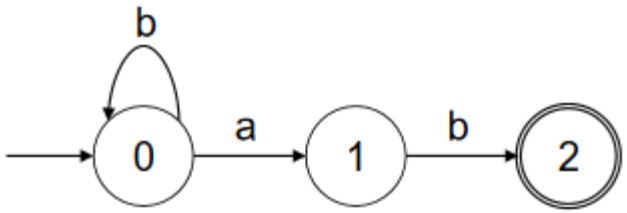
- Se o autômato é determinístico, então o seguinte algoritmo pode ser aplicado

```
s:=s0
c:=próximo_caractere()
enquanto (c<>eof) faça
      s:=transição(s,c)
      c:=próximo_caractere()
fim
se s for um estado final
   então retornar "cadeia aceita"
   senão retornar "falhou"
```

Implementação do Analisador Léxico – Solução: Métodos Dirigidos por Tabela

```
s:=s0
c:=próximo_caractere()
enquanto (c<>eof) faça
      s:=transição(s,c)
      c:=próximo_caractere()
fim
se s for um estado final
   então retornar "cadeia aceita"
   senão retornar "falhou"
```

Estado/ Final	Símbolo de entrada		
	a	b	outro
0/não	1	0	---
1/não	---	2	---
2/sim	---	---	---



Implementação do Analisador Léxico – Solução: Métodos Dirigidos por Tabela

- Vantagem: elegância (código é reduzido) e generalidade (mesmo código para várias linguagens)
- Desvantagem: pode ocupar grande espaço quando o alfabeto de entrada é grande;
- Métodos dirigidos por tabela são usados em geradores como o Lex.

Implementação do Analisador Léxico – Solução Ad hoc

- Para implementar o analisador léxico mais facilmente, podemos agrupar os autômatos em um único autômato que seria o próprio analisador

```
procedimento analisador léxico;
{
    obteve_símbolo = FALSE;

    enquanto não obteve_símbolo {
        caracter = leia_caracter_nao_branco();

        símbolo = &;

        se caracter é LETRA então
            identificador();
        senão se caracter é DÍGITO então
            número();
        senão operador();
    }

    return( código, símbolo );
}
```

Implementação do Analisador Léxico – Solução Ad hoc

- Para implementar o analisador léxico mais facilmente, podemos agrupar os autômatos em um único autômato que seria o próprio analisador

```
procedimento identificador()
{
    enquanto caracter é LETRA ou DÍGITO {
        símbolo = símbolo || caracter;
        caracter = leia_caracter();
    }

    se tamanho( símbolo ) > MÁXIMO_PERMITIDO {
        escreva advertência;
        considere apenas o MÁXIMO_PERMITIDO;
    }

    se símbolo é palavra reservada então
        código = código_referente_à_palavra;
    senão
        código = código_de_identificador;

    obteve_símbolo = TRUE;
}
```

Implementação do Analisador Léxico – Solução Ad hoc

- Para implementar o analisador léxico mais facilmente, podemos agrupar os autômatos em um único autômato que seria o próprio analisador

```
procedimento numero()
{
    enquanto caracter é DÍGITO {
        símbolo = símbolo || caracter;
        caracter = leia_caracter();
    }

    se valor( símbolo ) < MÍNIMO_ACEITO {
        escreva advertência;
        considere apenas o MÍNIMO_ACEITO;
    senão se valor( símbolo ) > MÁXIMO_ACEITO {
        escreva advertência;
        considere apenas o MÁXIMO_ACEITO;
    }

    código = código_de_número;

    obteve_símbolo = TRUE;
}
```

Implementação do Analisador Léxico – Solução Ad hoc

- Para implementar o analisador léxico mais facilmente, podemos agrupar os autômatos em um único autômato que seria o próprio analisador

```
procedimento operador()
{
    obteve_símbolo = TRUE;

    se caracter = '+' {
        código = código_de_adição;
    } senão se caracter = '*' {
        caracter = leia_caracter();
        se caracter = '*' {
            código = código_de_exponenciação;
        } senão {
            código = código_de_multiplicação;
        }
    } senão se caracter = '=' {
        código = código_atribuição;
    } senão {
        escreva advertência;
        obteve_símbolo = FALSE;
    }
}
```